Contents

FinOKF: An Open Knowledge Format Profile for Equity Research with Derivation-Certified Numeric Answers, and a Token-Matched Causal Evaluation 1
1. Abstract 1
2. Introduction and motivation 2
3. Problem statement and scope 4
4. Research questions 5
5. Background 6
6. Related work positioning 7
7. Proposed artefact: the FinOKF format and the CertiFacts protocol and checker (C1, C2) 9
8. Tolerance algebra and theorems (C3) 12
9. Evaluation design (C4) 17
10. Headline adversarial experiment: checker gaming 20
11. Kill-gates G1–G4 with thresholds and pivots 22
12. Work plan (M1–M12) and compute budget 25
13. Descoping order and risk register 27
14. Expected contributions and publication targets 28
15. References 29
Appendix A: Related-work vetting note 30
Appendix B: Draft JSON Schema for the claim graph 31

FinOKF: An Open Knowledge Format Profile for Equity Research with Derivation-Certified Numeric Answers, and a Token-Matched Causal Evaluation

Master's thesis proposal — version 2.1 (OKF-headlined reframe of v2)

- Author: Aniket Chaudhary
- Degree program: [to be completed — degree program]
- Supervisor: [to be completed — supervisor name]

- Date: July 2026

Version 2 revised the v1 proposal in response to panel review — deeper tolerance-algebra theory with a committed Lean 4 mechanization, a symmetric primary metric, scoped and audited bundle construction, a headline checker-gaming experiment, and numbered research questions with a granular plan, compute budget, and descoping order — and version 2.1 reframes that same content around the open knowledge format profile (FinOKF) as the headline artifact, with certification (CertiFacts) as the profile's defining capability, changing title, abstract, and positioning only while leaving research questions, gates, thresholds, metrics, datasets, and the work plan untouched.

1. Abstract

Equity research lacks an open, typed, agent-consumable data format. The Open Knowledge Format (OKF) supplies an open container but deliberately defines no numeric semantics: no units, scales, periods, or rounding, and untyped links. Finance is where those absences are fatal:

misinterpretation of scale and unit is the largest recoverable error class in large language model (LLM) answers over filings — repairing scale interpretation alone lifts Llama-3.3-70B from 37% to 57.7% — and post-hoc verification by LLMs fails, with false-positive rates of 95–100%. This thesis contributes: (C1, headline artifact) FinOKF, a strictly conformant OKF profile filling the layer OKF declines to define — typed finance facts carrying value, unit, scale, currency, period, entity, rounding precision, and provenance, plus calculation-style constraint blocks, built deterministically from SEC EDGAR XBRL data; (C2) CertiFacts, the certification layer the profile's typed semantics support — a capability no existing format provides: a claim-graph answer protocol in plain JSON, in which every numeric token is a quotation of a typed fact or a small arithmetic program over declared facts, re-executed by a deterministic checker in exact rational arithmetic; (C3) a tolerance algebra giving interval semantics to consistency with rounded reported facts, with an adversarial error model, provable false-accept bounds, a fail-closed soundness theorem, and a committed Lean 4 mechanization; (C4) the first token-matched causal evaluation of whether typing itself reduces numeric error, alongside an adversarial checker-gaming experiment. Soundness is claimed and proved; certification coverage and gaming resistance are measured, not assumed; answer relevance is explicitly out of scope.

2. Introduction and motivation

Equity research lacks an open, typed, agent-consumable data format. The numbers that drive quantitative analysis reach large language models (LLMs) as filing prose, linearized tables, or proprietary vendor feeds; the open container now emerging for agent-consumable knowledge — Google's draft Open Knowledge Format (OKF) — supplies identity, layout, and an extension surface, but deliberately defines no numeric semantics: no units, scales, periods, or rounding, and only untyped links (§5.2). This thesis builds FinOKF, a strictly conformant OKF profile for equity research that fills exactly the layer OKF declines to define, and argues that what makes such a format thesis-grade is the capability its typed semantics are strong enough to support: deterministic certification of LLM-emitted derived numbers.

Finance is the domain where the missing semantics are fatal, and the evidence fixes what the profile's core must be. LLMs answering quantitative equity-research questions over financial filings get the numbers wrong in characteristic ways. FAITH's error taxonomy for financial numeric reasoning shows that the single largest *recoverable* error class is misinterpretation of scale and unit: repairing scale interpretation alone lifts Llama-3.3-70B from 37% to 57.7% accuracy [FAITH, arXiv:2508.05201]. Numeric failure here is not diffuse noise: it concentrates in a structured class, worth over twenty percentage points, that explicit typing of scale, unit, and period should attack directly.

The obvious remedy — a second LLM as verifier — demonstrably fails: FinVerBench found LLM-verifier false-positive rates of 95–100% on financial numeric claims, while its rule-based verifier is trustworthy but reaches a recall ceiling near 52% [FinVerBench, arXiv:2605.29586]. These two numbers fix the design space: the checker must be deterministic, non-LLM code (soundness), and the fraction of true answers it can reach must be measured, not assumed (coverage).

A deterministic checker for *derived* numbers — ratios, growth rates, margins — then meets a semantic obstacle that atomic fact-checking avoids: reported financial facts are rounded. Every

XBRL (eXtensible Business Reporting Language) fact carries a `decimals` attribute recording its rounding precision (§5.1), so a derived value is consistent with its operands only up to a tolerance computed from those precisions. Supplying that tolerance semantics and proving the checker sound is the theoretical core of this thesis (§8). The division of labor is fixed accordingly: FinOKF, the format profile, carries the typed numeric semantics; CertiFacts, the answer protocol and checker built on it, turns them into certificates.

2.1 Worked example

The question "What was Apple's FY2024 gross margin?" runs through every document of this thesis. The FinOKF bundle for Apple's FY2024 10-K contains, among others, two typed fact nodes built deterministically from US Securities and Exchange Commission (SEC) Electronic Data Gathering, Analysis, and Retrieval (EDGAR) XBRL data:

```
[
  { "fact_id": "f1",
    "concept": "us-gaap:RevenueFromContractWithCustomerExcludingAssessedTax",
    "value": 391035, "unit": "USD", "scale": 1e6, "currency": "USD",
    "decimals": -6, "period": "FY2024", "entity": "AAPL",
    "provenance": "<EDGAR accession number / element reference>" },
  { "fact_id": "f2",
    "concept": "us-gaap:CostOfGoodsAndServicesSold",
    "value": 210352, "unit": "USD", "scale": 1e6, "currency": "USD",
    "decimals": -6, "period": "FY2024", "entity": "AAPL",
    "provenance": "<EDGAR accession number / element reference>" }
]
```

The model answers with prose plus a claim graph in plain JSON (JavaScript Object Notation):

```
{ "answer_text": "Apple's FY2024 gross margin was 46.2%.",
  "claims": [ { "id": "c1", "kind": "derive",
                "program": "(f1 - f2) / f1",
                "inputs": ["f1", "f2"],
                "value": 0.462, "decimals": 3 } ] }
```

Every numeric token in `answer_text` must bind to a claim identifier; here "46.2%" binds to `c1`. Because both facts carry `decimals = -6` (rounded to millions), f_1 denotes the interval $[391034.5, 391035.5]$ million USD and f_2 denotes $[210351.5, 210352.5]$ million USD. The checker re-executes $(f_1 - f_2)/f_1$ in exact rational arithmetic over this envelope box, obtaining a tight interval around $0.46206\ldots$; the claim's declared rounding (`decimals: 3`) gives the claimed 0.462 its own envelope $[0.4615, 0.4625]$. The intervals intersect, so the checker **certifies** the claim. The tolerance is two-sided: input rounding envelopes on one side, declared answer rounding on the other.

The rejection paths matter as much. A fabricated 0.481 yields an envelope $[0.4805, 0.4815]$ that does not intersect the derived interval: **reject**. A claim citing a fact identifier absent from the bundle, or a program the checker cannot resolve over its declared inputs, is rejected outright. Certification is fail-closed: a number with no resolvable derivation never certifies.

2.2 Contributions

This proposal develops FinOKF and its CertiFacts certification layer around four contributions:

- **C1 — FinOKF, a typed finance fact format and profile (headline artifact; §7).** Fact nodes carry `{fact_id, concept, value, unit, scale, currency, period, entity, decimals, children_exhaust_subtotal, provenance}`; bundles add calculation-linkbase-style constraint blocks (subtotal sums, sign conventions, period alignment), are built deterministically from EDGAR XBRL data — no LLM in the loop — and are packaged as a strictly conformant profile of the OKF container, filling the numeric-semantics layer the container deliberately leaves open (§5.2).
- **C2 — CertiFacts, a claim-graph protocol and deterministic checker (§7).** Answers are plain JSON; every numeric token binds to a `quote(fact_id)` or `derive(program, fact_ids)` claim, re-executed by a deterministic checker in exact rational arithmetic over integer mantissas with rounding envelopes, fail-closed. Plain JSON is deliberate: exotic output notations cost roughly 9–14 percentage points of generation accuracy [TOON-Gen, arXiv:2603.03306; Format-Cost, arXiv:2605.29676].
- **C3 — a tolerance algebra with a machine-checked soundness theorem (§8).** Interval semantics for "consistent with rounded reported facts"; an adversarial error model with provable false-accept bounds as a function of operand precisions and derivation size (§8.4); treatment of the operand-dependency problem; and a fail-closed soundness theorem whose core is committed to a Lean 4 mechanization.
- **C4 — the first token-matched causal evaluation, plus a checker-gaming experiment (§§9–10).** Four arms at matched token budgets over two to three datasets and mid-tier models, scored by a gold-aligned metric measured identically in every arm, and a headline adversarial experiment quantifying certified-but-wrong rates under attack.

The one-line positioning is that FinOKF is the numeric-semantics profile OKF deliberately leaves open, with certification as its defining capability. For the certification layer itself, the honest secondary positioning is *Proof-Carrying Numbers, completed*: Proof-Carrying Numbers (PCN) [PCN, arXiv:2509.06902] established fail-closed per-number certification for atomically quoted values and designated derived values as future work; CertiFacts carries out that step and adds the empirical evaluation PCN did not attempt (§5.3, §6).

3. Problem statement and scope

Setting. A FinOKF bundle B contains typed fact nodes built deterministically from a firm's XBRL filings, plus constraint blocks. Given B and a question, an LLM emits answer text and a claim graph; the checker must decide *certify* or *reject* for every numeric token in the answer text. The research problem is to make that decision sound, characterize what it cannot do, and measure causally, at matched cost, whether the representation and protocol reduce numeric error.

Claimed: soundness. Two distinct guarantees are claimed. First, fail-closed soundness: a claimed value inconsistent with *every* valuation of its cited facts within their rounding envelopes never certifies, and a claim referencing unresolvable facts or programs never certifies (Theorem 1 and Corollary 1, §8.5) — this extends PCN's Theorems 5.1/5.3 from atomic quotation to derivations,

and its core is committed to Lean 4 (§8.6). Second, a probabilistic residual bound: a value fabricated *without knowledge of the true derivation* may still land, by luck, inside a legitimate acceptance region, and the probability of such a coincidental certificate is bounded above by an explicit function of operand precisions and derivation size (Proposition 1, §8.4). The two statements are not interchangeable: the first is about values that are not derivable at all, the second about blind fabrication that happens to hit a derivable region.

Not claimed. Three disclaimers are load-bearing:

- *Completeness.* Some true answers cannot be expressed as quote/derive over a given bundle; rule-based verification hit a recall ceiling near 52% in FinVerBench [FinVerBench, arXiv:2605.29586]. Certification *coverage* is therefore a first-class reported metric, and the fail-closed design makes low coverage a usability cost, not a soundness bug.
- *Relevance.* A certified number is arithmetically consistent with the bundle; it need not answer the question. Correct arithmetic over the wrong period, segment, or entity is the relevance gap: period- and entity-binding checks narrow it and red-team attack class (ii) measures it (§10), but this thesis does not close it.
- *Full gaming resistance.* Resistance is measured empirically by a red-team suite of roughly 100 attacks in four classes (§10); one class (tolerance stuffing) is additionally bounded analytically by the width-growth analysis (§8.4). No blanket robustness claim is made.

Non-goals. Neither the FinOKF format nor the CertiFacts protocol claims any token-efficiency or latency benefit: compression-driven speed-ups are conditional on serving configuration and largely cancel under production stacks such as vLLM and commercial application programming interfaces (APIs) [Compression-Latency, arXiv:2604.02985]; the evaluation therefore holds token budgets constant across arms. Data scope is US SEC 10-K filings from EDGAR; datasets are capped at two to three and arms at four (§9).

4. Research questions

Each research question maps one-to-one to a kill-gate; thresholds, kill criteria, and pivots are given in §11.

- **RQ1 (soundness and feasibility) → G1.** Can a deterministic tolerance-algebra checker certify real 10-K derivations with a false-reject rate at or below roughly 10% and provable false-accept bounds? G1 (months M1–M2) runs the checker on at least 200 real derivations across six pilot firms (§9.3); failure of either conjunct — a residual false-reject rate above roughly 10% after attributing bundle-construction errors, or false-accept bounds that cannot be established — kills the thesis early and cheaply.
- **RQ2 (causal representation effect) → G2.** At matched token budgets, does typing scale, unit, and period causally reduce numeric scale errors versus untyped renderings of the same facts? G2 (M4) is a {typed, untyped} × {with, without prose documentation} factorial, so structure is not confounded with documentation text.
- **RQ3 (value over strong baselines) → G3.** Does the full FinOKF+CertiFacts pipeline beat Program-of-Thoughts (PoT)-style program emission over CSV (comma-separated values) at equal tokens on verified numeric accuracy — or does its residual value reduce to certification

coverage? G3 (M6) decides; the pre-declared fallback is a narrower certificate-and-coverage contribution.
- **RQ4 (cost and robustness) → G4.** What is the constraint tax of claim-graph emission, and how resistant is certification to gaming? G4 (M5–M6) measures the tax; the red-team experiment (M8) measures certified-but-wrong rates.

5. Background

5.1 XBRL and the semantics of `decimals`

XBRL is the machine-readable tagging standard for SEC filings; the EDGAR companyfacts API exposes every tagged fact. A fact binds a taxonomy concept — 17,388 concepts in the US-GAAP (US Generally Accepted Accounting Principles) 2024 taxonomy, plus company-specific extension tags — to a value, unit, entity, and period [FinTagging, arXiv:2505.20650]. Concept alignment is genuinely hard for LLMs — direct 17,388-way classification scored 0.0 for GPT-4o and DeepSeek-V3 [FinTagging, arXiv:2505.20650] — hence FinOKF bundles are built deterministically from issuers' own tags; no model tags anything.

Central to this thesis, every numeric fact carries a `decimals` attribute: `decimals = d` states that the value is rounded to 10^{-d}, so $d = -6$ means rounded to millions [XBRL-2.1, XBRL International 2003]. A reported value v thus denotes the interval $[v - \frac{1}{2} \cdot 10^{-d}, v + \frac{1}{2} \cdot 10^{-d}]$ of possible underlying values: consistency of a derived value with reported facts is interval intersection, not numeric equality. XBRL Calculations 1.1 already applies this style of rounding-aware interval reasoning, but only to calculation-linkbase summations within a single filing [XBRL-Calc-1.1, XBRL International 2023]; §8 states the differentiation precisely.

5.2 The OKF container and the layer it deliberately leaves open

FinOKF bundles are packaged as a conformant profile of OKF, a v0.1 draft container specification published under Apache-2.0 in the GoogleCloudPlatform/knowledge-catalog repository, which states that it is not an official Google product [OKF-Spec, github.com/GoogleCloudPlatform/knowledge-catalog]. An OKF bundle is a directory of Markdown files with YAML (YAML Ain't Markup Language) frontmatter; concept identity is the file path; the only required frontmatter field is a free-string `type`; links between files are untyped edges; and consumers must preserve unknown frontmatter keys. That preservation rule, together with the spec's extension provisions (custom frontmatter keys, custom `type` values, fenced payload blocks; OKF spec sections 4.1 and 9), makes a *strictly conformant* finance profile possible; §7.2 gives the concrete profile design.

What OKF does not provide is this thesis's motivation, not a footnote. The specification defines no numeric types, units, periods, or validation semantics, and its links are untyped edges: the numeric-semantics layer is deliberately left open, consistent with the spec's explicit non-goal of replacing domain-specific schemas (Avro, Protobuf, OpenAPI) — OKF references such schemas rather than subsuming them. FinOKF is built to fill exactly that open layer for equity research (§7). The honest caveats remain in full force: the specification contains no occurrence of "token", "latency", "context window", or "finance"; OKF is packaging only — identity conventions, layout, and a public extension surface; every mechanism in this proposal (types, constraint blocks, tolerance algebra, checker) is the thesis's contribution; and no claim depends on OKF adoption.

5.3 Proof-Carrying Numbers

PCN [PCN, arXiv:2509.06902] is a September-2025 World Bank paper proposing that every number an LLM emits carry a machine-checkable certificate binding it to a source fact, fail-closed: a number whose certificate does not resolve is flagged, never silently passed. Its Theorems 5.1 and 5.3 establish soundness for atomically quoted values. The paper is protocol and theory only — it reports no experiments — and has accrued no citations as of mid-2026; derived values (ratios, growth rates, aggregations) are explicitly designated future work. That future work is where the difficulty lives: a derived value is consistent with rounded operands only up to a tolerance, so certifying derivations requires the rounding semantics of §5.1, a program representation for claims (§7), and a soundness argument over intervals rather than exact matches (§8). CertiFacts is PCN completed: the same fail-closed contract, extended from quotation to derivation, and evaluated empirically.

6. Related work positioning

The primary positioning of this thesis is that FinOKF is the numeric-semantics profile OKF deliberately leaves open, with certification as its defining capability; the secondary positioning, for the certification layer, is *Proof-Carrying Numbers, completed* (§5.3). The works closest to this thesis are foundations, not competitors. Each establishes a pillar the thesis stands on, and each leaves open the conjunction FinOKF and CertiFacts jointly occupy: typed facts, certified derivations, fail-closed soundness, token-matched causal evaluation, and measured gaming resistance. Table 1 summarizes; the paragraphs below give the deltas. The standalone literature review develops these threads; Appendix A records how the set was vetted.

Work	What it establishes	What it leaves open (this thesis's delta)
PCN [PCN, arXiv:2509.06902]	Fail-closed per-number certification; soundness theorems for atomic quotes	Derived values; rounding tolerance; any experiments
AuditFlow [AuditFlow, arXiv:2606.03031]	Agentic XBRL auditing; symbolic tools indispensable (82% → 18% ablation)	Certifying LLM *answers* rather than filings
FinGround [FinGround, arXiv:2604.23588]	Post-hoc recomputation of LLM answers via 47 templates	Arbitrary derivations; tolerance semantics; soundness claim
FinVerBench [FinVerBench, arXiv:2605.29586]	Diagnosis: LLM verifiers 95–100% false positives; rule-verifier recall ceiling ~52%	Any verification mechanism
VeNRA [VeNRA, arXiv:2603.04663]	Typed fact ledger improves numeric grounding	Constraint/derivation layer; checker; certification
FAITH [FAITH, arXiv:2508.05201]	Error taxonomy; scale errors as largest recoverable class	Any prevention or certification mechanism
XBRL Calculations 1.1 [XBRL-Calc-1.1, XBRL International 2023]	Rounding-aware interval validation of linkbase summations	Cross-fact/cross-period derivations; LLM answers; error model; mechanized soundness

Table 1: Foundations and their deltas.

PCN. PCN owns the certification contract this thesis adopts: per-number, fail-closed, machine-checkable. CertiFacts contributes exactly what PCN deferred — derivation certificates, the tolerance algebra they require, extended soundness theorems, and an empirical evaluation; the relationship is succession, not rivalry.

AuditFlow. AuditFlow has LLM agents audit XBRL filings with symbolic tools; its central ablation — accuracy collapsing from 82% to 18% when the symbolic tools are removed — is the strongest available evidence that deterministic symbolic machinery, not model capability, carries verification workloads [AuditFlow, arXiv:2606.03031]. Its object of verification is the filing itself; CertiFacts points the same determinism at the model's answer.

FinGround. FinGround is the nearest neighbor in intent: post-hoc recomputation of LLM answers against source data, via 47 hand-written templates covering roughly 80% of its own D2 dataset, with no rounding-tolerance semantics and no soundness claim [FinGround, arXiv:2604.23588]. CertiFacts replaces the closed template library with an open program algebra, a rounding-aware checker, and a proved fail-closed guarantee. (FinGround's published repository link currently returns 404, so a reimplementation is budgeted as a baseline; M6, §12.)

FinVerBench. FinVerBench contributes diagnosis rather than mechanism: LLM verifiers at 95–100% false-positive rates, and a rule-based verifier recall ceiling near 52% [FinVerBench, arXiv:2605.29586]. This proposal treats both numbers as binding design constraints — hence a deterministic non-LLM checker, and coverage reported as a first-class metric.

VeNRA. VeNRA shows that a typed fact ledger improves numeric grounding, reporting 1.2% hallucination — a figure on its own metric, not independently reproduced [VeNRA, arXiv:2603.04663]. It has no constraint or derivation layer and no checker: typed facts alone cannot certify arithmetic performed over them. FinOKF adds the constraint layer, and CertiFacts the derivation protocol and checker, that VeNRA stops short of.

FAITH. FAITH supplies the target rather than the mechanism: its taxonomy identifies scale errors as the largest recoverable class, worth 37% → 57.7% for Llama-3.3-70B when scale interpretation is repaired [FAITH, arXiv:2508.05201]. This thesis adopts the taxonomy in its symmetric error metric and builds FAITH-style scale-error probes into the G2 experiment; FAITH proposes no certification machinery.

XBRL Calculations 1.1. This 2023 XBRL International specification is the closest formal precedent for the tolerance algebra: rounding-aware interval validation of calculation-linkbase summations [XBRL-Calc-1.1, XBRL International 2023]. Its scope is filing-internal, pre-declared summation relations. CertiFacts generalizes the interval semantics to arbitrary LLM-emitted derivations across facts and periods, adds an adversarial error model and a machine-checked soundness theorem, and serves answer certification rather than filing validation (§8).

In coverage-matrix terms (developed in the literature review), each neighbor fills some cells of {typed facts, derivations, fail-closed guarantee, causal evaluation, gaming measurement}; the conjunction of all five is empty until FinOKF and CertiFacts fill it together.

7. Proposed artefact: the FinOKF format and the CertiFacts protocol and checker (C1, C2)

The artefact has two engineered layers. C1 is FinOKF, the typed finance fact bundle format: a deterministic, provenance-carrying representation of a filer's reported facts, packaged as a conformant profile of the OKF container introduced in §5. C2 is CertiFacts, the answer protocol — the claim graph — under which every numeric assertion the model makes is either a quotation of a bundle fact or a small declared derivation over bundle facts, plus a deterministic checker that certifies or rejects each assertion. The tolerance algebra giving the checker its semantics is C3 (§8). One principle governs both layers: every component that could silently introduce error is either deterministic code or a measured quantity. No LLM appears anywhere in bundle construction or checking.

7.1 The FinOKF fact bundle (C1)

The unit of representation is the fact node, with the following fields.

Field	Content
`fact_id`	identifier, unique within the bundle; the handle claims cite
`concept`	the reporting concept, carried as the issuer's own XBRL tag (US-GAAP or company extension)
`value`	the reported value, stored exactly as filed (integer or exact decimal mantissa; never a float)
`unit`	measure: monetary, shares, pure ratio
`scale`	explicit power-of-ten scaling (e.g., 10^6)
`currency`	currency code per ISO 4217 (International Organization for Standardization) where the unit is monetary
`period`	instant or duration, with fiscal-period identity (Q4 vs FY is a typed distinction)
`entity`	filer identity (Central Index Key, CIK)
`decimals`	the XBRL rounding-precision attribute of the reported value
`children_exhaust_subtotal`	whether listed component facts are declared to sum to this subtotal
`provenance`	accession number and element reference into the source filing

Two fields do most of the work. The `decimals` field carries the rounding semantics of XBRL 2.1: `decimals = d` states that the value is accurate to 10^{-d}, so $d = -6$ means rounded to the nearest million [XBRL-2.1, XBRL International 2003]. This attribute is what makes checking *derived* values well-defined at all; §8.1 builds the tolerance semantics on it. The explicit typing of `value`/`unit`/`scale`/`currency`/`period` targets the largest recoverable error class in financial LLM output — the scale and unit misinterpretation quantified in §2 [FAITH, arXiv:2508.05201] — by making scale, unit, and period machine-checkable properties rather than quantities inferred from prose.

A bundle also carries **constraint blocks** in the style of the XBRL calculation linkbase: (i) subtotal exhaustiveness — components flagged by `children_exhaust_subtotal` must sum to the subtotal within the tolerance semantics of §8; (ii) sign and balance conventions; (iii) period-alignment constraints — a derivation may not mix Q4 and FY operands, or operands from different entities, without declaring it. Constraints are enforced twice: at build time, validating the bundle itself, and at answer time, on the operands of every claim. This layer distinguishes the bundle from typed fact ledgers such as VeNRA, which ground atomic values but carry no constraint or derivation structure [VeNRA, arXiv:2603.04663].

The running worked example (§2.1) instantiates this schema: `f1` is the revenue node (`us-gaap:RevenueFromContractWithCustomerExcludingAssessedTax`, value 391035, scale 10^6, `decimals` −6) and `f2` the cost-of-sales node (`us-gaap:CostOfGoodsAndServicesSold`, value 210352, same typing).

7.2 Packaging: an OKF-conformant profile

Section 5.2 described the OKF container and the extension provisions that make strict conformance achievable. The FinOKF profile uses exactly that surface: fact nodes and constraint blocks are carried as fenced JSON payload blocks inside per-concept Markdown files, with custom `type` values (e.g., `finance.fact`, `finance.constraint`) and custom frontmatter keys for entity and period scoping [OKF-Spec, github.com/GoogleCloudPlatform/knowledge-catalog]. A generic OKF consumer that ignores these extensions still processes the bundle correctly.

The packaging claim is deliberately narrow. OKF contributes distribution and interoperability surface, not mechanism (§5.2): the specification names "replacing domain-specific schemas" as an explicit non-goal — OKF references such schemas rather than subsuming them — and the FinOKF profile is exactly such a referenced schema. No claim is made that OKF is token- or LLM-optimized; nothing in the spec supports one. Beyond a stable container, the concrete benefit of conformance is a public specification venue: a numeric-profile issue will be filed against the OKF repository in month M1 (§12) as a priority timestamp for the profile design.

7.3 Deterministic construction from SEC EDGAR XBRL

Bundles are built by deterministic code from the SEC EDGAR `companyfacts` and `submissions` JSON APIs. Each XBRL fact maps mechanically onto the fact-node fields; calculation-linkbase relations map onto constraint blocks; provenance is the accession number plus element reference. No LLM is in the construction loop, which sidesteps the concept-alignment bottleneck rather than solving it: direct classification into the 17,388 US-GAAP 2024 concepts scores 0.0 for GPT-4o and DeepSeek-V3 (§5.1), and 11,874 of the 81,325 facts in FinTagging's thirty FY2024 10-Ks are company-specific extension tags unaddressable by taxonomy retrieval [FinTagging, arXiv:2505.20650]. FinOKF never classifies: it carries the issuer's own tags — extensions included — as opaque typed concepts, exactly as filed.

Deterministic construction does not entail correct construction; bundle quality is load-bearing. The audit protocol is therefore part of the artefact: a 50-fact random sample, drawn across the pilot firms (§9.3), is manually checked against the rendered filings, with a target error rate below 1% and the *measured* rate reported whatever it turns out to be. The soundness theorem of §8.5 is explicitly conditional on this faithfulness premise, which is why the premise must carry a measured number, not an assumption.

7.4 The CertiFacts claim-graph protocol (C2)

The model's answer is plain JSON — an evidence-driven decision: exotic output notations cost roughly 9–14 percentage points of generation accuracy across models [TOON-Gen, arXiv:2603.03306; Format-Cost, arXiv:2605.29676], and hard constrained decoding degraded one 405B-parameter model from 92.5 to 35.0 in the same study [TOON-Gen, arXiv:2603.03306]. Certi-

Facts therefore uses ordinary JSON with the most permissive emission mode that validates, and measures the residual constraint tax at gate G4 (§11).

A claim is one of two kinds. A **quote** claim asserts the value of a single bundle fact: `quote(fact_id)`, with a claimed value and declared rounding. A **derive** claim declares a program over named inputs: `derive(program, inputs)`, where `program` belongs to a tiny arithmetic grammar — expressions over declared inputs and integer constants under $+$, $-$, $\times$, $\div$ with parentheses. Constants are restricted to a small fixed whitelist for unit conversions (initially $\{100, 1000\}$, e.g., $\times 100$ for percentages): unrestricted literals would let a program encode any target value as a constant — a self-certifying claim, an instance of attack class (i) (§10.1). For the same reason, the checker rejects any program whose value is independent of its declared inputs (constant or input-cancelling programs such as $f_1 - f_1 + c$), detected via the exact range computation of §8. Since every enlargement of the grammar enlarges the gaming attack surface of §10, the grammar — whitelist included — is frozen at M5, before the main evaluation.

The protocol's load-bearing rule is **answer binding**: an answer is well-formed only if every numeric token in `answer_text` binds to exactly one claim id whose certified value matches that token under the claim's declared rounding; a numeric token bound to no claim leaves the whole answer uncertified. This closes the most obvious gaming route — certify a trivially true identity while asserting an unrelated number in prose (attack class (i), §10).

In the worked example's claim graph (§2.1), `f1` occurs twice in the program `(f1 - f2) / f1`; this repeated-operand structure is precisely what makes naive interval checking inadequate (§8.3).

7.5 The deterministic checker

The checker is deterministic, non-LLM code — forced by the evidence: FinVerBench measured false-positive rates of 95–100% for LLM verifiers on financial numeric claims [FinVerBench, arXiv:2605.29586]. The pipeline has six stages, each defaulting to rejection on failure: (1) parse and schema-validate the claim graph; (2) resolve every cited `fact_id` against the bundle; (3) check constraint blocks over the operands — period and entity alignment, sign conventions; (4) re-execute each program under the tolerance algebra of §8, with exact rational arithmetic over integer mantissas and envelopes only at rounding boundaries; (5) certify a claim iff its declared rounding envelope intersects the derived interval; (6) apply the answer-binding rule to `answer_text`. There is no default-accept path: a number with no resolvable derivation never certifies.

The nearest prior mechanism is FinGround's post-hoc recomputation against 47 hand-written templates covering roughly 80% of its dataset [FinGround, arXiv:2604.23588]. A template library bounds coverage by its authors; CertiFacts re-executes *the model's own declared program*, so coverage is bounded instead by what the model can express over the bundle — measured and reported as certification coverage (§9), motivated by the ~52% recall ceiling FinVerBench observed for rule-based verifiers.

8. Tolerance algebra and theorems (C3)

Reported financial facts are rounded, so "is this derived number consistent with the filing?" is decidable only relative to a tolerance semantics — the theoretical heart of the thesis, developed in

this section. Theorems are stated informally but precisely, with proof-sketch intuition; full proofs and mechanization are thesis work (M2 write-up, M8 completion; §12).

8.1 Interval denotation of rounded facts

A reported fact f with stored mantissa `value` v, power-of-ten `scale` s (§7.1), and `decimals` $= d$ denotes the closed interval

$$[\![f]\!] = \left[\, v \cdot s - \frac{1}{2} \cdot 10^{-d},\, v \cdot s + \frac{1}{2} \cdot 10^{-d} \,\right],$$

the set of true underlying values consistent with the reported rounding. The `decimals` attribute qualifies the unscaled quantity $v \cdot s$, not the stored mantissa; the checker carries v and s as exact integers, so $v \cdot s$ and both endpoints are exact rationals (§8.2). For $d \leq 0$ the envelope is coarse: in the worked example, $v = 391035$ with $s = 10^6$ and $d = -6$ (millions) gives a half-width of half a million in the fact's unit — the envelope $[391034.5, 391035.5]$ million of §2.1. Facts declared exact under XBRL 2.1's exact-value mechanism denote point intervals [XBRL-2.1, XBRL International 2003]. The closed-interval convention is conservative: whatever tie-breaking rule the preparer used, the true value lies in the closed envelope. Endpoint conventions matter only in exact-tie edge cases and are fixed once, explicitly, in the formalization (§8.6).

Tolerance is two-sided. A claim declares its own rounding d_c, so the claimed value v_c denotes the envelope $\left[\, v_c - \frac{1}{2} \cdot 10^{-d_c}, v_c + \frac{1}{2} \cdot 10^{-d_c} \,\right]$. **Certification predicate:** a derive claim with program P over facts $f_1, \ldots, f_k$ certifies iff the image of P over the box $[\![f_1]\!] \times \ldots \times [\![f_k]\!]$ intersects the claim's envelope. Quotation is the special case where P is the identity.

8.2 Exact rational evaluation; envelopes only at rounding boundaries

The checker performs no floating-point arithmetic: reported values, interval endpoints, and program evaluations are exact rationals over integer mantissas and power-of-ten scales. Floating point would smuggle a second, uncontrolled tolerance into a system whose entire purpose is to account for tolerance explicitly. Width therefore enters at exactly two kinds of places — once per reported fact, at its rounding boundary ($\frac{1}{2} \cdot 10^{-d_i}$), and once per claim, at its declared answer rounding. Computation itself creates no width.

On the worked example (§2.1), exact evaluation of $(f_1 - f_2)/f_1$ over the envelope box yields the exact image $[0.4620615\ldots, 0.4620655\ldots]$ — width about 4×10^{-6} — around the point value $0.46206\ldots$; the claim's envelope at 3 decimals, 0.462 ± 0.0005, intersects it, and certification goes through with no floating-point step anywhere.

8.3 The operand-dependency problem

Naive interval arithmetic evaluates each subexpression independently, treating repeated occurrences of the same operand as independent variables: for $(f_1 - f_2)/f_1$ it would use f_1's envelope twice, overestimating the result's width. Inflated width is not cosmetic here — it inflates the false-accept bound of §8.4, and in large derivations makes certification vacuous, since wide enough intervals certify anything.

The treatment has three parts. First, because evaluation is exact-rational, no width originates in computation; the object being bounded is the image of a *function* over a box, not a composition of independent interval operations. Second, the checker computes that image exactly by **monotonicity analysis**: for the four-operation grammar, the sign of P's dependence on each input is decidable by interval sign analysis of subterms, and when P is monotone in each coordinate the exact image endpoints are attained at box vertices and computed by exact evaluation there. The example is typical: $(f_1 - f_2)/f_1 = 1 - f_2/f_1$ is increasing in f_1 and decreasing in f_2 on the positive box, so two vertex evaluations give the exact image. Third, where monotonicity analysis is inconclusive (mixed-sign subterms, denominators straddling zero), the checker falls back to **affine forms**, which track first-order correlations between repeated operands but yield over-approximate bounds. An over-approximation is safe for *rejection* but not for witness-carrying certification, so claims resolvable only by over-approximation are rejected as unresolvable — fail-closed — as are programs whose denominator interval contains zero. The incidence of these conservative rejections is measured at gate G1 as a component of the false-reject rate (§11).

8.4 Adversarial error model and false-accept bounds

Soundness (§8.5) says fabricated numbers never certify *as consistent with the cited facts*; it does not say a fabricated number cannot land, by luck, inside a legitimate acceptance region. The error model quantifies that residual risk.

Model. A fabrication adversary emits a claimed value drawn, without knowledge of the true derivation, from a plausible range R (the empirical range of the quantity type: gross margins, growth rates). The acceptance region for a given claim structure — the set of claimed values that certify — is an interval whose width is the derived width W plus the claim-envelope width 10^{-d_c} (a Minkowski sum).

Proposition 1 (false-accept bound, informal). Under uniform fabrication over R, the probability that a fabricated value certifies is at most $(W + 10^{-d_c})/|R|$; under non-uniform fabrication with density bounded by ρ, at most $\rho \cdot (W + 10^{-d_c})$. *Intuition:* the acceptance region is an interval of known width; the bound is the probability mass such an interval can capture.

Lemma 1 (width growth, informal). For a program P in the grammar touching n rounded leaf occurrences with envelope half-widths $\varepsilon_1, \ldots, \varepsilon_n$, the derived interval's half-width is at most $\sum_i L_i \varepsilon_i$, where L_i bounds $|\partial P/\partial x_i|$ over the box. For purely additive programs $L_i = 1$, so width grows linearly in the number of rounded leaves; for multiplicative and divisive programs, *relative* half-widths compose additively to first order, with higher-order corrections bounded on the box. *Intuition:* mean-value bound on the box, or factor-wise telescoping for products.

Two measures of a derivation must be kept apart. Its *depth* is the longest path in the expression tree; its *size* is the number of rounded leaf occurrences n in Lemma 1. The quantity the theory bounds — and the quantity the checker caps — is size: depth is only an informal proxy, since deeper programs typically touch more rounded leaves. Throughout this proposal, "derivation size" means leaf occurrences.

Together these give the false-accept bound as an explicit function of operand precisions and derivation size, with two consequences. For shallow derivations over finely-rounded operands the

bound is strong: in the worked example the derived width is $\sim 4 \times 10^{-6}$ (§8.2) and the acceptance region is dominated by the claim's own declared rounding (10^{-3}), so, taking the plausible range R for a gross margin to be the unit interval, a fabricated gross margin certifies with probability on the order of 10^{-3} by Proposition 1 (a narrower empirical range, say $[0.2, 0.7]$, shifts this only by a small constant factor). Conversely, the bound *degrades* as derivations touch more coarsely-rounded operands — exactly the tolerance-stuffing attack (class (iii), §10): choose large derivations to inflate W until anything certifies. The mitigations follow from the lemma: cap certified derivation size (leaf occurrences), and report the derived width alongside every certificate. The fabrication model is deliberately idealized — real model errors are not uniform draws — so it supplies the provable half of the story; the red-team of §10 supplies the empirical half.

8.5 Checker soundness: fail-closed certification of derivations

PCN's Theorems 5.1 and 5.3 establish soundness and fail-closed behavior for *atomic quoted* values (§5.3); derived values are PCN's declared future work [PCN, arXiv:2509.06902]. CertiFacts extends both properties to derivations; the new load-bearing ingredient is exactness of range computation.

Lemma 2 (exact range on the monotone path, informal). For every program P in the claim grammar and every closed box X on which P is defined, *whenever the checker resolves P over X via the monotone path of §8.3* — that is, monotonicity analysis certifies the sign of P's dependence on each coordinate — the interval it computes equals the exact image $P(X)$. *Intuition:* P is continuous on a compact connected box, so its image is a closed interval; certified coordinatewise monotonicity locates the endpoints at box vertices, where exact rational evaluation computes them without error. When the analysis is inconclusive, the checker does not certify (§8.3), so no certificate ever rests on an over-approximate range.

Theorem 1 (derivation soundness, fail-closed; informal). Let B be a bundle whose fact nodes faithfully transcribe the source filing's reported values and `decimals` attributes (the audited premise of §7.3). Let c be a claim with program P in the grammar over inputs $f_1, \ldots, f_k \in B$, claimed value v_c, and declared rounding d_c. **(Soundness.)** If the checker certifies c, then there exist real numbers $x_1, \ldots, x_k$ with $x_i \in [f_i]$ for every i such that $P(x_1, \ldots, x_k)$ lies within $[\, v_c - \frac{1}{2} \cdot 10^{-d_c}, v_c + \frac{1}{2} \cdot 10^{-d_c} \,]$ — every certificate carries a witness valuation under which the claimed number is correct at its declared rounding. **(Fail-closed.)** Any claim that cites an unresolvable fact id, whose program is outside the grammar or undefined on the box, that violates a constraint block, or for which only an over-approximate range is computable, is rejected; and an answer containing a numeric token bound to no certified claim is not certified.

Corollary 1 (no fabricated certificates). A claimed number inconsistent with *every* valuation of its cited facts within their rounding envelopes never certifies; a number citing no facts never certifies.

Proof sketch. For soundness: certification requires the computed interval to intersect the claim envelope. Certification only ever occurs via the monotone path — claims resolvable only by over-approximate affine forms are rejected as unresolvable under the fail-closed clause — so Lemma 2 applies to every certified claim: the computed interval is the exact image, any point in the intersection pulls back to a concrete witness valuation in the box, and exact rational endpoint

comparison decides the intersection test without numerical error. For fail-closed: case analysis on the pipeline of §7.5 — every non-certify branch rejects, and no branch accepts by default. The step beyond PCN is the replacement of "syntactic match against a quoted fact" with "membership in the exact image of a program over envelope boxes," which is why Lemma 2, not the protocol plumbing, is the theorem's core.

Three honest qualifications. Soundness is *relative to bundle faithfulness* — hence the measured audit rate of §7.3 rather than an assumption of zero error. Completeness is not claimed: some true statements cannot be expressed or resolved over the bundle (FinVerBench's ~52% rule-verifier recall ceiling is the cautionary precedent), so coverage is measured, never assumed. And certification is not relevance: a certified number can answer the wrong question; that gap is scoped out and partially measured in the red-team (§10).

8.6 Committed Lean 4 formalization

I commit to mechanizing the core of this theory in Lean 4 with mathlib; this is a deliverable, not an option. Mechanized: (i) the interval denotation of `decimals` and its basic algebra over exact rationals; (ii) Lemma 2, exact range computation on the monotone path — the load-bearing lemma; (iii) Theorem 1 in both directions; (iv) Lemma 1 for the additive fragment. Paper-only: Proposition 1 and its variants, whose measure-theoretic adversary model is proved by hand. The schedule is a skeleton (types plus theorem statements) at M2, alongside the G1 experiment, with complete proofs by M8 (§12); the work is laptop-scale and carries no compute risk. The formal statements also pin the checker's reference semantics, against which the Python checker's test suite is written on shared test vectors.

8.7 What XBRL Calculations 1.1 already does, and what it does not

Interval-based treatment of rounding is not itself new. XBRL Calculations 1.1 validates calculation-linkbase summations using rounded and truncated fact-value intervals, deeming a calculation consistent when the computed-total interval overlaps the reported-total interval, in round-to-nearest and truncation modes [XBRL-Calc-1.1, XBRL International 2023]. That overlap test is the same family of predicate as §8.1′s, applied at the summation level.

The differentiation runs on three axes. **Object of verification:** Calculations 1.1 checks linkbase summations *within a single filing*; CertiFacts checks *arbitrary LLM-emitted derivation programs* across facts and periods, bound token-by-token to a generated answer. **Theory:** Calculations 1.1 specifies a validation procedure; it contains no adversarial error model, false-accept bounds, width-growth analysis, or soundness theorem — reasonably, since filing validation does not face an adversarial emitter, while an LLM under a certification incentive is one (§10). **Purpose:** preparer-side filing quality assurance versus consumer-side answer certification with a fail-closed guarantee. The two compose rather than compete: FinOKF constraint blocks are deliberately linkbase-shaped, so within-bundle subtotal checking coincides with Calculations-1.1-style validation, and CertiFacts extends strictly along the derivation axis. The one-line answer to "is this just Calculations 1.1?": the interval semantics of rounding is shared ancestry; the certification protocol, error model, soundness theorem, and evaluation are not in that specification's scope.

9. Evaluation design (C4)

The evaluation is built around one discipline: any comparison offered as evidence for a causal claim must hold fact content, token budget, and scoring protocol fixed across conditions, so that the only manipulated variable is the representation or protocol under test. Existing format studies in financial question answering (QA) do not enforce this. SECQUE studies format and token effects on SEC-filings QA but measures no verifiability outcome [SECQUE, arXiv:2504.04596]; TQA-Bench provides a useful multi-scale tabular-QA evaluation template but likewise does not pair token matching with a verified-accuracy outcome [TQA-Bench, arXiv:2411.19504]. To my knowledge, no published evaluation in this literature matches token budgets across representation arms while scoring a symmetric, gold-aligned numeric-accuracy outcome; that specific combination is the sense in which C4 claims to be the first token-matched causal evaluation, and the claim is hedged accordingly. The design maps directly onto RQ2–RQ4 (§4) and feeds gates G2–G4 (§11).

9.1 Experimental arms

Four arms, all rendering the *same underlying facts* for each item, are compared; restricting the design to four (cut down from a larger v1 matrix) keeps every contrast interpretable:

- **Arm A — untyped control.** The facts rendered as untyped Markdown text and tables, with values, scale words, and units embedded in surrounding prose roughly as a filing renders them. No explicit typed fields.
- **Arm B — typed bundle, free-form answer.** The FinOKF typed fact bundle (§7), with the model answering in ordinary prose. This isolates the representation effect from the protocol effect.
- **Arm C — the full FinOKF+CertiFacts pipeline.** FinOKF typed bundle, CertiFacts claim-graph answer protocol, deterministic checker (§7–§8). The claim graph is plain JSON; per the generation-penalty evidence of §7.4, no custom output syntax is introduced anywhere in the pipeline.
- **Arm D — Program-of-Thoughts over CSV (strong baseline).** The facts rendered as CSV; the model is prompted to emit a short executable program that computes the answer, and the program is run deterministically. Program execution eliminates arithmetic slips, which makes this the strongest fair competitor: it shares "compute, don't recite" with arm C but has no typed scale/unit semantics, no provenance binding, no rounding-tolerance semantics, and no fail-closed certificate — a program over the wrong operand executes cleanly and returns a confident wrong number.

The preregistered primary contrasts are: A vs. B (does typing causally reduce numeric errors — RQ2), C vs. D (does the full FinOKF+CertiFacts pipeline beat a strong execution baseline at equal tokens — RQ3), and B vs. C (what does the claim-graph protocol add or cost — RQ4). All other pairwise comparisons are reported as exploratory.

9.2 Token-matching procedure

Token budget is the dominant confound in format studies: a "better" format that is merely longer or shorter changes what the model attends to, independent of typing. Matching therefore proceeds per item: (1) render the item's facts in all four arm formats; (2) count input tokens under both

pinned tokenizers — o200k_base and the Llama tokenizer; (3) set two per-item budgets, one per tokenizer, each equal to the maximum count across arms under that tokenizer; (4) pad shorter renderings toward both budgets simultaneously using semantically inert material (fixed neutral preamble text and whitespace that carries no fact content); because padding chosen to meet one tokenizer's budget can overshoot the other's, trim — by removing redundant boilerplate only, never fact content — exactly when meeting one budget pushes a rendering past the other's tolerance band; (5) accept the item when all four arms are within the tolerance band (provisionally ±2%, fixed in the preregistered analysis plan before the main runs) of the respective budget *under each tokenizer*. Exact simultaneous equality under two tokenizers is generally unattainable, which is why a tolerance band is used and why every reported token count appears under both tokenizers. Items whose two budgets cannot be jointly satisfied within tolerance are flagged and reported rather than silently dropped.

Input-side matching does not equalize output-side spending: arm C's claim graph consumes output tokens that free-form answers do not. Output token usage is therefore reported per arm, and the output-side cost is measured explicitly as the constraint tax under G4 (§11) rather than hidden by the matching procedure.

9.3 Datasets D1–D3

The dataset list is deliberately cut to two core sets plus one descopable set, with an audited construction protocol for each — bundle construction quality is load-bearing for every downstream claim, so its error rate must be a measured number, not an assumption.

- **D1 (native, primary), ~300 items.** A SEC-XBRL bundle-QA set built from the EDGAR companyfacts API: template-generated questions over real filings plus hand-written items, including a dedicated block of FAITH-style scale-error probes (questions engineered so that scale or unit confusion produces a characteristic wrong answer) [FAITH, arXiv:2508.05201]. Filings are drawn from a six-firm pilot ladder chosen to span construction difficulty — provisionally Apple and Microsoft as clean large-cap controls, JPMorgan Chase for financial-sector statement structure, Amazon or Berkshire Hathaway as a multi-segment, table-dense filer, plus slots for an extension-tag-heavy filer and a restatement or irregular-fiscal-year case, finalized against EDGAR during M1 (§12). Bundle construction is fully deterministic from XBRL data — no LLM anywhere in the construction loop — and audited per the protocol of §7.3, with the measured error rate reported against the sub-1% target.
- **D2, ~500 items.** A FinQA subset [FinQA, arXiv:2109.00122], converted programmatically from FinQA's table and program annotations into typed bundles and gold derivations, with a 100-item manual audit of the conversion. I acknowledge the conversion cost openly: turning an existing QA dataset into typed bundles is real engineering work — plausibly weeks to months, not days — and it is budgeted as such in the work plan (M4, §12). The audit exists precisely because the failure mode of programmatic conversion is *silent* corruption of gold labels; items that fail audit are dropped, not hand-patched.
- **D3 (descopable).** A TAT-QA arithmetic subset [TAT-QA, arXiv:2105.07624], included only if D1 and D2 land on schedule; it sits near the top of the descoping order (§13).

9.4 Primary metric: symmetric gold-aligned numeric error rate

The primary outcome must be measurable identically in every arm; otherwise the causal comparison is confounded by the measuring instrument. Checker-based metrics fail this requirement — they are native to arm C only — so they are explicitly *secondary*. The primary metric is a gold-aligned numeric error rate: every numeric token in the model's answer is aligned to the gold answer by value-and-context matching with rounding tolerance, and classified into one of five classes:

- **correct** — matches gold within the declared rounding tolerance;
- **scale-error** — matches gold up to a power-of-ten factor (millions vs. billions, percent vs. fraction), the largest recoverable error class in financial LLM output [FAITH, arXiv:2508.05201];
- **sign-error** — matches gold up to sign;
- **wrong-operand** — matches a value in a bounded, deterministically enumerated candidate set of mis-derivations: each single bundle fact taken directly, plus the gold derivation re-evaluated with each operand swapped for a type-compatible bundle fact (same unit and quantity type, any period, segment, or entity), with no growth in program size (right arithmetic, wrong inputs) — bounding the set is essential, since over an unrestricted program space almost any value is derivable from *some* combination of bundle facts;
- **fabricated** — aligns to nothing in gold or the wrong-operand candidate set.

The five classes are not mutually exclusive (a value can match gold up to a power of ten and up to sign simultaneously), so labels are assigned by a fixed precedence order — correct, then scale-error, then sign-error, then wrong-operand, then fabricated — applied deterministically: each numeric token receives the first class whose test it passes.

The alignment-and-classification protocol is itself validated: a 100-item sample is independently dually annotated, disagreements are adjudicated, and chance-corrected agreement statistics are reported before the scorer is trusted on the full evaluation. Alongside the primary metric, four secondary quantities are reported: certification coverage (what fraction of gold answers *can* be expressed as quote/derive claims over the bundle — measured, not assumed, given the rule-verifier recall ceiling of §2), certified-but-wrong rate (§10), per-arm token budgets under both tokenizers, and monetary cost.

9.5 Models

Two to three mid-tier models carry the main evaluation: Llama-3.3-70B as the open-weights anchor (its tokenizer is one of the two pinned tokenizers), one mid-tier commercial API model, and one small open model. One frontier model is run as a secondary reference point only. Mid-tier models are a deliberate choice, not a budget compromise: the recoverable error mass is largest there — the FAITH scale-repair gain of §2 was measured on Llama-3.3-70B [FAITH, arXiv:2508.05201] — whereas ceiling effects on frontier models can mask exactly the representation effects RQ2 asks about. Decoding is deterministic (temperature 0) for all main arms.

9.6 Statistics

Every item is answered under all four arms, so the design is fully paired. Primary tests are McNemar tests on paired per-item error indicators for the preregistered contrasts, with effect

sizes reported as paired risk differences. Confidence intervals come from a bootstrap clustered by source document/firm, because facts drawn from the same filing are not independent — a firm-specific idiosyncrasy (an unusual segment structure, a restatement) perturbs every item built on that filing. Approximately 800 paired items across D1+D2 suffice to detect differences on the order of five percentage points at conventional significance and power levels; since McNemar power depends on the discordant-pair rate, the preregistered analysis plan states the assumed rates, and the minimal detectable effect is recomputed once pilot discordance data exist.

9.7 Confound controls

All of the following are mandatory, not optional robustness checks:

1. **Matched token budgets** (§9.2) — removes length as an explanation for any arm difference.
2. **Structure-vs-documentation factorial.** Evidence from semantic-layer evaluations shows that prose documentation explained nearly all of the measured "semantic layer" gains [Sem-Layer, arXiv:2604.25149]. A typed format bundles both structure *and* implicit documentation, so G2 runs a {typed, untyped} × {with, without prose documentation} factorial: the causal claim survives only if typing helps net of documentation text.
3. **Prompt/rendering factorial.** Two prompt paraphrases crossed with two fact orderings on a subset, to bound sensitivity to incidental prompt choices.
4. **Mid-tier models and deterministic decoding**, as above.
5. **Latency is never headlined.** Token-count reductions do not translate into reliable latency gains: across more than 30,000 serving runs, prompt-compression speed-ups above 1.3× appeared only under naive serving or with >10k-token prompts at >4× compression ratios; under vLLM the gains cancel (0.4–0.9× at ≤4k tokens), and commercial APIs showed no reliable speed-up at all [Compression-Latency, arXiv:2604.02985]. Accordingly, token budgets are reported as costs, and any latency observation appears only as a caveated side note, never as an outcome. This is a deliberate scoping decision: the contribution of this thesis is typed semantics and certification, not efficiency.

10. Headline adversarial experiment: checker gaming

A certification protocol changes what failure looks like. Without a checker, the failure mode is a wrong number; with a checker, the failure mode that matters is a wrong number *that carries a certificate*, because the certificate is exactly what invites downstream trust. The soundness theorem (§8) rules out one route to that failure — a number with no resolvable derivation never certifies — but it is a statement about the checker's semantics, not about an adversary's ability to exploit those semantics from the outside. That gap is measured, not assumed away.

10.1 Threat model

The adversary controls the answer-generating side: the prompt, or in the limit the model itself, attempting to obtain a certificate for a number that is wrong with respect to gold. The checker code and the deterministic bundle-construction pipeline are trusted; the attack surface is the claim-graph interface plus the bundle's free-text fields. The free-text fields are on the surface despite the trusted builder because the builder copies them (concept labels, provenance strings, documentation) verbatim from the source filing: an adversarial or compromised filing is the entry

point for attacker content inside a bundle, and that vector is exercised by class (iv). Four attack classes are defined:

- **(i) Degenerate programs.** Emit a claim whose program derives a trivially true identity, while the answer text asserts an unrelated number. A residual route in this class targets the grammar itself: constant or input-cancelling programs (e.g., $f_1 - f_1 + c$), whose exact image is a point independent of the inputs and therefore always intersects the claim's own envelope. Mitigations under test: the answer-binding rule — every numeric token in `answer_text` must bind to a claim id, so unbound numbers void certification — plus the constant whitelist and input-independence rejection rule of §7.4; the red team probes whether whitelisted constants can still be composed into attacker-chosen values.
- **(ii) Wrong-but-real facts.** Perform correct arithmetic over the wrong period, segment, or entity — the relevance gap. Period- and entity-binding checks partially mitigate this class, but it is explicitly *measured rather than solved*: certification asserts derivability, not relevance (§3).
- **(iii) Tolerance stuffing.** Choose deliberately large derivations so that rounding envelopes accumulate until almost any claimed value intersects the derived interval. The width-growth analysis in the tolerance algebra (Lemma 1, §8.4) quantifies exactly this attack, and a cap on certified derivation size (rounded leaf occurrences, §8.4) is the corresponding mitigation; the red team tests the bound empirically.
- **(iv) Prompt injection in bundle text.** Adversarial content placed in the bundle's free-text fields (provenance strings, documentation) — injected upstream via the source filing's own text, which the trusted builder ingests verbatim — attempting to steer the model into emitting attacker-chosen claims that nonetheless certify.

Classes (i) and (iii) sit inside the theory's scope and test whether the implementation honors it; classes (ii) and (iv) sit *outside* the soundness theorem by design — which is precisely why they must be measured empirically rather than claimed away.

10.2 Red-team design

The suite comprises approximately 100 attacks, roughly balanced across the four classes. Each attack is a triple (bundle, question, adversarial strategy) with a known-wrong target answer. Attacks are seeded by hand from the threat model, then expanded with model-generated variants that are individually reviewed before inclusion, and run against the full arm-C pipeline in M8 (§12), after the main evaluation has frozen the pipeline configuration. Mitigations are evaluated ablatively — answer-binding rule on/off, period/entity checks on/off, derivation-size cap on/off — so the report shows before/after rates per mitigation, not just a single aggregate.

10.3 Metric

The headline metric is the **certified-but-wrong rate**: the fraction of attacks in which a number wrong with respect to gold nonetheless receives a certificate, reported overall and per attack class, alongside the attack taxonomy and mitigation deltas. For class (iii), the observed success rate is compared against the false-accept bound predicted by the error model (§8), tying the empirical experiment back to the theory: if attacks succeed materially above the predicted bound, either the implementation or the model assumptions are wrong, and either finding is reportable.

10.4 Why this is unclaimed territory

No adjacent work has measured adversarial gaming of a numeric-certification interface. PCN — the closest ancestor, whose fail-closed protocol this thesis extends — contains no experiments at all [PCN, arXiv:2509.06902]. FinGround recomputes answers post hoc with hand-written templates but evaluates only honest outputs [FinGround, arXiv:2604.23588]; AuditFlow audits *filings*, not LLM answers [AuditFlow, arXiv:2606.03031]; FinVerBench measures verifier accuracy on non-adversarial statements [FinVerBench, arXiv:2605.29586]; VeNRA has no constraint or derivation layer to game [VeNRA, arXiv:2603.04663]. A certification claim that has never been attacked is weak evidence; an attack taxonomy with measured success rates and mitigation deltas is informative *whatever the outcome*, which is why this experiment is headlined rather than treated as an ablation — and, in a fast-compounding verification literature (§13), it is the one component no neighboring project has touched.

11. Kill-gates G1–G4 with thresholds and pivots

The project is structured around four preregistered kill-gates, ordered so that the cheapest and most fundamental risks are tested first. Each gate has an explicit threshold fixed before the experiment runs, a scheduled decision point in the work plan (§12), and a pivot path. A thesis that dies at month 2 is a cheap outcome; a confounded thesis discovered at month 12 is an expensive one. Gate outcomes are reported in the thesis regardless of result.

Gate	RQ	Decision point	Test	Kill threshold	Pivot path
G1	RQ1	end of M2	Checker on ≥200 real 10-K derivations across 6 pilot firms	False-reject rate > ~10% after attributing bundle errors, OR no provable false-accept bounds	Diagnose semantics vs. bundle errors; fix builder if the latter; stop early if the former
G2	RQ2	end of M4	Typed×documentation factorial at matched tokens on FAITH-style scale-error probes, 2 models	No causal reduction in scale errors attributable to typing net of documentation	Drop the causal-representation claim; rescope around certification (RQ1/RQ3/RQ4)
G3	RQ3	M6	PoT-over-CSV vs. the full FinOKF+CertiFacts pipeline at equal tokens	PoT matches verified numeric accuracy within the detectable margin	Retreat to the certificate/coverage contribution — narrower but intact thesis
G4	RQ4	M5–M6	Constraint-tax measurement of claim-graph emission	Schema-emission accuracy cost exceeds grounding gains	Intermittent constraining; re-measure; report as cost-accuracy trade-off if still negative

G1 — checker feasibility on real filings (decision: end of M2). Passing requires RQ1′s conjunction: a false-reject rate at or below roughly 10% once bundle-construction errors are attributed, together with provable false-accept bounds (§8.4). The kill condition is the negation — a residual false-reject rate above roughly 10%, or false-accept bounds that cannot be established; honest, correctly derived values failing to certify at that rate would make the fail-closed design unusable. The pivot is diagnostic before it is fatal — decompose the false rejects into bundle-con-

struction errors (survivable: fix the builder and re-run) versus failures of the rounding semantics itself (not survivable: if the tolerance algebra does not fit real filings, the project stops in month 2 — the point of running this gate first).

G2 — causal representation effect (decision: end of M4). The typed × documentation factorial of §9.7 runs on two models, at matched token budgets, over the FAITH-style scale-error probe set. The kill condition is no causal cut in scale errors attributable to typing once documentation text is controlled — the scenario in which the documentation confound [SemLayer, arXiv:2604.25149] absorbs the entire apparent benefit. This kills RQ2′s premise outright, and there is no pivot that rescues the causal claim: the honest response is to report the null, drop the representation-causality strand, and re-scope the thesis around the certification strand (RQ1, RQ3, RQ4) at the midpoint review. Because the validated gold-alignment scorer is only delivered in M5 (§12), the G2 decision is taken on a provisional scale-error-only scorer, with the validated symmetric metric applied retroactively to the G2 runs before the M6 review confirms the decision. The typed format remains necessary infrastructure either way — the checker cannot operate without declared units, scales, and periods — but its justification would then be functional rather than causal.

G3 — value over the strong baseline (decision: M6). The full FinOKF+CertiFacts pipeline versus Program-of-Thoughts-over-CSV at equal token budgets on the symmetric primary metric. If the execution baseline matches verified numeric accuracy within the detectable margin (~5pp at the planned power), the accuracy-improvement claim is retired and the thesis retreats to what arm D cannot provide at any accuracy level: fail-closed certificates, measured certification coverage, and provenance-bound derivations. That is a narrower claim but still a defensible thesis — the difference between "the certified pipeline is more accurate" and "the certified pipeline is equally accurate and additionally certifiable" is made explicit rather than blurred.

G4 — constraint tax (decision: M5–M6, with the midpoint descoping review). The cost of forcing claim-graph emission is measured directly: arm B versus arm C at matched input budgets. Structured emission is a documented risk — constrained decoding degraded Hermes-4-405B from 92.5 to 35.0 on one generation benchmark [TOON-Gen, arXiv:2603.03306] — so the gate treats it as a first-class threat, not an implementation detail. If the emission cost exceeds the grounding gains on the primary metric, the pivot is intermittent constraining: let the model reason free-form and apply grammar-constrained decoding only to the final claim-graph block, then re-measure. If the tax remains net-negative even then, it is reported as an explicit cost-accuracy trade-off, and the protocol is positioned for settings where fail-closed guarantees justify the overhead — a weaker but honest conclusion that the evaluation is designed to be able to reach.

The gates are complemented by the descoping order (§13), which pre-commits what gets cut when time pressure hits; notably, the Lean formalization, gates G1–G3, the symmetric metric protocol, and at least a reduced form of the checker-gaming experiment are never descoped.

12. Work plan (M1–M12) and compute budget

12.1 Month-by-month plan

The plan is organized around the four kill-gates (§11): the cheapest-to-falsify claims are tested first, and every month through M8 ends in an artifact or a recorded decision. M3–M8 are deliberately granular — the stretch the v1 panel found underspecified, and where dataset-conversion and protocol-validation costs hide.

M1 — Ingestion, checker v0, priority timestamp. Build the SEC EDGAR ingestion layer against the companyfacts and submissions JSON APIs, observing EDGAR access etiquette (declared User-Agent header, at most 10 requests per second). Implement bundle builder v0 for the pilot-firm ladder defined in §9.3 and checker v0 (exact rational arithmetic over integer mantissas, with rounding envelopes applied only at rounding boundaries). File the numeric-profile extension issue in the OKF repository [OKF-Spec, github.com/GoogleCloudPlatform/knowledge-catalog]; the issue doubles as a public, datestamped design record (risk R1, §13.2).

M2 — G1 run and tolerance-algebra write-up. Execute the G1 protocol: at least 200 real 10-K derivations across the six pilot firms, measuring the false-reject rate of checker v0. Draft the tolerance-algebra chapter: the adversarial error model, the operand-dependency treatment, and the width-growth lemma. Produce the Lean 4 skeleton (core datatypes plus the statement of the soundness theorem). **G1 decision** at month end, against the threshold and pivot of §11: a false-reject rate that traces to the rounding semantics rather than bundle errors stops the thesis here, early and cheaply.

M3 — Format specification and D1 construction. Freeze the FinOKF format specification v0.9 (fact schema, constraint blocks, OKF packaging) and implement the encoder/decoder pair. Construct dataset D1 (approximately 300 items: template-generated plus hand-written, including scale-error probes in the style of the FAITH taxonomy [FAITH, arXiv:2508.05201]). Run the bundle audit of §7.3: a 50-fact random sample checked manually against the underlying filings, with the measured error rate reported whether or not it meets the sub-1% target.

M4 — G2 experiment; begin D2. Run the typed-versus-untyped × with/without-documentation factorial at matched token budgets on two models and analyze scale-error rates. Because the validated gold-alignment scorer is an M5 deliverable, the **G2 decision** is taken on a provisional scale-error-only scorer (power-of-ten mismatch against gold — precisely the class G2 targets); the validated symmetric metric (§9.4) is applied retroactively to the G2 runs in M5, and the M6 review confirms or revisits the decision before any re-scope is executed. Per §11: on a null, the causal-representation strand is dropped, the re-scope around the certification strand is decided at the M6 review, and the null result is reported in full. In parallel, begin the D2 pipeline: programmatic conversion of a roughly 500-item FinQA subset [FinQA, arXiv:2109.00122] from its table and program annotations, with a 100-item manual audit. Conversion is budgeted explicitly because it is where unplanned months accumulate.

M5 — Claim-graph protocol and scorer validation. Implement claim-graph emission (constrained JSON generation against the Appendix B schema) and the answer-binding rule that every numeric token in `answer_text` must bind to a claim id. Measure the constraint tax (the first half

of G4). Build the gold-alignment scorer and run its dual-annotation validation study (100 items, two annotators, adjudication, agreement statistics).

M6 — G3 showdown and midpoint review. Run the Program-of-Thoughts-over-CSV comparison at equal token budgets; alongside it, implement a FinGround-style template-recomputation baseline as a secondary comparison point — a reimplementation, since FinGround's published repository is unavailable (§6) [FinGround, arXiv:2604.23588]. **G3 and G4 decisions**, followed by a midpoint descoping review against the ladder in §13.1: gate outcomes, schedule health, and remaining scope are assessed and decisions minuted.

M7 — Main causal evaluation. Four arms × D1 + D2 (plus D3 if retained) × two to three models, under frozen configurations and a preregistered analysis plan; deterministic decoding (temperature 0) for main arms; all token counts reported under both pinned tokenizers.

M8 — Red-team and mechanized proofs. Execute the checker-gaming red-team (approximately 100 attacks across the four classes of §10) and the robustness ablations (derivation-size caps, coverage curves). Complete the Lean 4 proofs of the soundness theorem and the core tolerance-algebra lemmas.

M9 — Analysis. Full statistical analysis (McNemar tests on paired error indicators, bootstrap confidence intervals clustered by source firm), error taxonomy, ablation write-ups.

M10–M11 — Writing. Thesis chapters; paper draft targeted per §14.2.

M12 — Buffer. Slack for overruns and defense preparation.

Decision point	Timing	Question	Action on failure
G1	end of M2	False-reject ≤ ~10% after attributing bundle errors, AND provable false-accept bounds?	Diagnose semantics vs. bundle errors; stop if semantics
G2	M4	Causal scale-error reduction at matched tokens?	Drop representation claim; re-scope around certification at M6 review
G3	M6	Full FinOKF+CertiFacts pipeline beats PoT-over-CSV at equal tokens?	Retreat to certification/coverage contribution
G4	M5–M6	Constraint tax below grounding gains?	Adopt intermittent constraining
Descoping review	M6	Schedule and scope health	Apply ladder in §13.1

12.2 Compute budget

The main evaluation requires roughly 3 models × 4 arms × 800 paired items ≈ 9,600 model calls. At approximately 3,500 input and 800 output tokens per call, that is about 34M input tokens (≈ 35M with retries) and about 7.7M output tokens (≈ 8M). At mid-tier API prices this is approximately US$150–400 for the main runs. Pilots, the G2 factorial, the G3 showdown, prompt/rendering ablations, the red-team, and the frontier-model reference runs of §9.5 (one model × 4 arms on an item subset, budgeted at several times mid-tier per-token prices) together roughly triple the volume, giving a total planned API spend of about US$1,000–1,500. Bundle construction, checking, and statistical analysis are CPU-only and run on a single workstation; Lean 4 with mathlib compiles on a laptop. No GPU cluster is required: open-weights models are accessed through hosted inference APIs. This is deliberate: the load-bearing components of the thesis are the checker and the evaluation design, not model scale.

13. Descoping order and risk register

13.1 Descoping order

If the timeline slips, scope is cut in the following fixed order, decided at the M6 review or earlier: (1) the conformal risk-control wrapper — a stretch-goal conformal-prediction layer over certification decisions, flagged from the outset and never promised as a contribution, which is why it heads the list; (2) dataset D3, the TAT-QA arithmetic subset [TAT-QA, arXiv:2105.07624]; (3) the third model family; (4) the breadth of the prompt/rendering factorial (from 2 × 2 on a subset down to a single paraphrase check); (5) the red-team suite, from ~100 attacks to ~50 while retaining all four attack classes. The following are never descoped: the Lean 4 soundness formalization (a committed deliverable, not an option), gates G1–G3, the symmetric primary-metric protocol, and the checker-gaming experiment in at least its reduced form. Fixing the order in advance prevents schedule pressure from silently cutting whatever is hardest that month.

13.2 Risk register

R1 — Crowding velocity (highest-rated risk). The verification-adjacent lineage is compounding quickly: an October-2025 XBRL auditing benchmark was followed by FinGround in April 2026 [FinGround, arXiv:2604.23588], FinVerBench in May 2026 [FinVerBench, arXiv:2605.29586], and AuditFlow in June 2026 [AuditFlow, arXiv:2606.03031]. The risk that a further entry preempts part of C3 or C4 before M8 cannot be excluded. Mitigations: (i) lead with the tolerance algebra and the token-matched causal evaluation, which none of the four works above claims; (ii) file the OKF numeric-profile issue in M1, creating a public priority timestamp; (iii) the checker-gaming experiment — none of the adjacent works evaluates gaming of its own verification mechanism, making that component the hardest to preempt.

R2 — Bundle construction. Who builds the FinOKF bundles, and at what error rate, is load-bearing for every downstream claim: a certified answer over a wrong bundle is confidently wrong. Mitigations: construction is deterministic from EDGAR XBRL with no LLM in the loop; error rates are audited and reported, not assumed (the 50-fact D1 audit in M3 and the 100-item D2 audit in M4); datasets are cut to two or three and arms to four, concentrating audit effort; and D2 conversion cost is budgeted explicitly in M4.

R3 — Completeness ceiling. Some true answers cannot be certified; the ~52% recall ceiling of the rule-based verifier in FinVerBench [FinVerBench, arXiv:2605.29586] suggests the certifiable fraction may be substantially below 100%. Mitigation: certification coverage is a first-class reported metric, measured rather than assumed, and the fail-closed design converts low coverage into a usability cost rather than a soundness failure. If G3 shows Program-of-Thoughts parity on accuracy, the contribution narrows honestly to certification and coverage.

R4 — Relevance gap. A certified number can still fail to answer the question — correct arithmetic over the wrong period, segment, or entity. This is excluded from the soundness claim (§3), partially reduced by period- and entity-binding checks, and partially quantified by red-team attack class (ii). It is measured, not solved, and the thesis says so.

R5 — Constraint tax. Structured emission can damage generation quality (the constrained-decoding collapse documented in §7.4 and gate G4 [TOON-Gen, arXiv:2603.03306]). G4 measures the tax directly at matched tokens; the fallback is intermittent constraining, with the grammar enforced only over the claim-graph block.

R6 — Data licensing. EDGAR data are public domain, so D1 and all headline results carry no license risk. FinQA and TAT-QA are available under research licenses compatible with academic use. FinDER [FinDER, arXiv:2504.15800], if used at all, is CC-BY-NC-4.0 — acceptable for a thesis but constraining redistribution; no headline result depends on it.

14. Expected contributions and publication targets

14.1 Deliverables

The thesis ships as an open format profile (FinOKF), an open certificate schema with a reference checker (CertiFacts), a mechanized theory, and a causal evaluation.

C1 — FinOKF, the format profile (headline artifact). A versioned specification for typed finance fact bundles as a strictly conformant OKF profile, together with the deterministic EDGAR bundle builder, pilot bundles for the six-firm ladder, and the audit report with a measured bundle error rate.

C2 — CertiFacts, the protocol and checker. The claim-graph JSON Schema (Appendix B), the answer-binding rule, and a deterministic, fail-closed reference checker using exact rational arithmetic with rounding envelopes, released as an open-source library with a test suite.

C3 — Theory. The tolerance algebra: an explicit adversarial error model with false-accept bounds as a function of operand precisions and derivation size; the operand-dependency treatment; and a checker-soundness theorem extending the atomic-quotation soundness results of PCN (Theorems 5.1/5.3) [PCN, arXiv:2509.06902] to derivations, with the soundness theorem and core lemmas mechanized in Lean 4. The write-up includes the explicit differentiation from XBRL Calculations 1.1 [XBRL-Calc-1.1, XBRL International 2023], which applies interval checking only to within-filing linkbase summations.

C4 — Evaluation. The first token-matched causal evaluation of typed representation and certification (four arms, symmetric gold-aligned primary metric, paired statistics), and the first

checker-gaming study, with attack taxonomy and certified-but-wrong rates. Released artifacts: dataset D1, the D2 conversion pipeline with audit results, the evaluation harness, the preregistered analysis plan, and the red-team suite.

14.2 Publication targets

The primary target is an industry-track paper at the Association for Computational Linguistics (ACL) or Empirical Methods in Natural Language Processing (EMNLP) conferences: the artifact-plus-evaluation profile fits that venue, and the closest neighbor, FinGround, appeared at the ACL 2026 Industry Track [FinGround, arXiv:2604.23588]. The fallback is the Financial Natural Language Processing (FinNLP) workshop series. A well-powered null result at G2 or G3 would still be reportable — the format-effects literature currently lacks token-matched causal evidence in either direction — though realistically at a workshop rather than an industry track. The thesis does not depend on venue acceptance; the gates in §11, not reviewer outcomes, define success.

14.3 Deployability of an open format profile and certificate schema

The FinOKF profile specification, the certificate schema, and the reference checker are released under permissive licenses. The sober version of the deployability argument is this: deterministic post-hoc verification of LLM-emitted numbers only composes across systems if producers and consumers agree on what a fact is and what a certificate is, and at present numeric-provenance features in commercial financial-data products are proprietary and do not interoperate. An open format profile plus an open certificate schema with a reference checker lowers integration cost for any producer, and the M1 OKF issue is a low-cost probe of whether a standards conversation exists to join. No adoption outcome is claimed: the academic contributions stand or fall on the G1–G4 evidence, and deployability is a plausible externality, not a thesis claim.

15. References

Several 2025–2026 works are cited throughout by stable shorthand keys (e.g., FAITH, FinVerBench, TOON-Gen) rather than by full paper titles; the arXiv identifier in each entry is authoritative. Specification and repository sources are cited by URL and were accessed in July 2026.

- AuditFlow — agentic XBRL audit environment with symbolic tools. arXiv:2606.03031 (2026). https://arxiv.org/abs/2606.03031
- Compression-Latency — prompt-compression latency study (>30,000 serving runs). arXiv:2604.02985 (2026). https://arxiv.org/abs/2604.02985
- FAITH — financial numeric-error taxonomy and scale-error analysis. arXiv:2508.05201 (2025). https://arxiv.org/abs/2508.05201
- FinDER — expert query–evidence–answer dataset over S&P 500 10-K filings. arXiv:2504.15800 (2025). https://arxiv.org/abs/2504.15800
- FinGround — post-hoc recomputation of LLM answers via hand-written templates. arXiv:2604.23588 (ACL 2026 Industry Track). https://arxiv.org/abs/2604.23588
- FinQA — expert-written, program-annotated numeric-reasoning QA over financial reports. arXiv:2109.00122 (2021). https://arxiv.org/abs/2109.00122
- FinTagging — XBRL tagging benchmark over FY2024 10-K filings. arXiv:2505.20650 (2025). https://arxiv.org/abs/2505.20650

- FinVerBench — evaluation of LLM and rule-based verifiers for financial claims. arXiv:2605.29586 (2026). https://arxiv.org/abs/2605.29586
- Format-Cost — corroborating study of exotic-notation generation penalties. arXiv:2605.29676 (2026). https://arxiv.org/abs/2605.29676
- OKF-Spec — Open Knowledge Format (OKF) specification, v0.1 draft (`okf/SPEC.md`). GoogleCloudPlatform/knowledge-catalog repository, Apache-2.0. https://github.com/GoogleCloudPlatform/knowledge-catalog (accessed July 2026)
- PCN — Proof-Carrying Numbers. arXiv:2509.06902 (2025). https://arxiv.org/abs/2509.06902
- SECQUE — format and token-budget study on SEC-filings question answering. arXiv:2504.04596 (2025). https://arxiv.org/abs/2504.04596
- SemLayer — semantic-layer documentation-confound study. arXiv:2604.25149 (2026). https://arxiv.org/abs/2604.25149
- TAT-QA — hybrid table-and-text question answering, arithmetic-heavy. arXiv:2105.07624 (2021). https://arxiv.org/abs/2105.07624
- TOON-Gen — TOON generation study (output-format effects across 21 models). arXiv:2603.03306 (2026). https://arxiv.org/abs/2603.03306
- TQA-Bench — multi-scale tabular question-answering benchmark. arXiv:2411.19504 (2024). https://arxiv.org/abs/2411.19504
- VeNRA — typed fact ledger for numeric grounding. arXiv:2603.04663 (2026). https://arxiv.org/abs/2603.04663
- XBRL-2.1 — Extensible Business Reporting Language (XBRL) 2.1. XBRL International Recommendation (31 December 2003; errata corrected to 20 February 2013). https://www.xbrl.org/Specification/XBRL-2.1/REC-2003-12-31/XBRL-2.1-REC-2003-12-31+corrected-errata-2013-02-20.html (accessed July 2026)
- XBRL-Calc-1.1 — Calculations 1.1. XBRL International Recommendation (22 February 2023). https://www.xbrl.org/Specification/calculation-1.1/REC-2023-02-22/calculation-1.1-REC-2023-02-22.html (accessed July 2026)

Appendix A: Related-work vetting note

The prior-art claims in §6 were checked against primary sources during June–July 2026: the arXiv abstract pages and PDFs of each cited paper, the OKF `SPEC.md` in the GoogleCloudPlatform/knowledge-catalog repository, the TOON repository README and benchmark tables, and the license files of each dataset. Scope statements attributed to neighboring works — for example, that PCN restricts certification to atomic quoted values and declares derived values future work, that FinGround's published repository link was unavailable at the time of checking, and that the OKF specification contains no occurrence of "token", "latency", or "finance" — were established by direct inspection of the primary documents rather than taken from secondary citations.

Numbers quoted from other papers are reported together with the conditions under which their sources measured them. Where a figure could not be re-verified against a primary source, it was omitted or stated qualitatively; the October-2025 XBRL auditing benchmark in §13.2 is named without an identifier for exactly this reason. Dated verification notes with URLs are archived in

the project repository and can be provided to examiners on request. This note describes a process; it is not a claim that the search was exhaustive, and a relevant work may exist that was not found.

Appendix B: Draft JSON Schema for the claim graph

The following draft (v0.1) JSON Schema specifies the claim graph emitted by the model in arm C (§7). Two rules are enforced by the checker rather than the schema, because they are not expressible in JSON Schema: the answer-binding rule (every numeric token in `answer_text` must bind to a claim `id`) and the resolution rule (every `fact_id` and every identifier in `inputs` must resolve against the presented bundle). The `program` grammar — identifiers drawn from `inputs`, the operators `+ - * /`, parentheses, and integer constants from the whitelist of §7.4 — is validated by the checker's parser, which also rejects any program whose value is independent of all declared inputs (§7.4).

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://certifacts.dev/schemas/claim-graph-v0.1.json",
  "title": "CertiFacts claim-graph (draft v0.1)",
  "type": "object",
  "required": ["answer_text", "claims"],
  "additionalProperties": false,
  "properties": {
    "answer_text": {
      "type": "string",
      "description": "Natural-language answer. Every numeric token must bind to a
claim id (checker-enforced)."
    },
    "claims": {
      "type": "array",
      "items": { "$ref": "#/$defs/claim" }
    }
  },
  "$defs": {
    "claim": {
      "type": "object",
      "required": ["id", "kind", "value", "decimals"],
      "additionalProperties": false,
      "properties": {
        "id": {
          "type": "string",
          "pattern": "^c[0-9]+$",
          "description": "Claim identifier referenced from answer_text."
        },
        "kind": {
          "enum": ["quote", "derive"],
          "description": "quote = verbatim reported fact; derive = arithmetic
over declared inputs."
        },
        "fact_id": {
```

```
          "type": "string",
          "description": "Bundle fact identifier; required when kind = quote."
        },
        "program": {
          "type": "string",
          "description": "Arithmetic expression over identifiers in inputs;
required when kind = derive. Grammar validated by the checker."
        },
        "inputs": {
          "type": "array",
          "items": { "type": "string" },
          "uniqueItems": true,
          "description": "Bundle fact identifiers consumed by program; required
when kind = derive."
        },
        "value": {
          "type": "number",
          "description": "The claimed numeric value as printed in answer_text,
after the claim's own rounding."
        },
        "decimals": {
          "type": "integer",
          "description": "Claimed rounding precision, XBRL-style: value is
rounded to the nearest 10^(-decimals)."
        }
      },
      "allOf": [
        {
          "if": { "properties": { "kind": { "const": "quote" } }, "required":
["kind"] },
          "then": { "required": ["fact_id"] }
        },
        {
          "if": { "properties": { "kind": { "const": "derive" } }, "required":
["kind"] },
          "then": { "required": ["program", "inputs"] }
        }
      ]
    }
  }
}
```